本地部署yolo训练环境

本地部署yolo训练环境
- 环境配置
- 下载并安装Yolo v5环境依赖
- 准备训练数据
- 使用YOLOv5训练水果分类模型
- 将训练的模型转为K230可以识别的kmodel
  - 下载转换工具
- 其它注意事项

环境配置

本节例程我们来演示一下在本地部署Yolo v5环境，并训练一个分类（识别）模型。

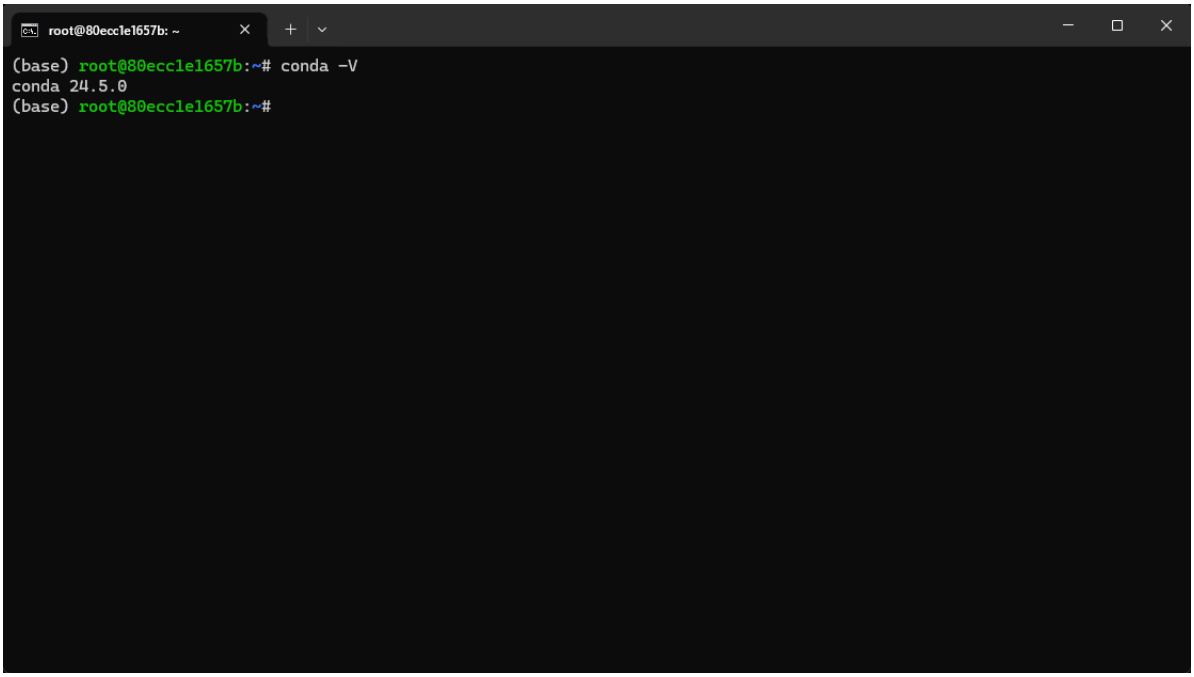
我们演示的操作环境如下：

系统：Ubuntu 22.04

GPU：4090D*1

CUDA版本： 12.4

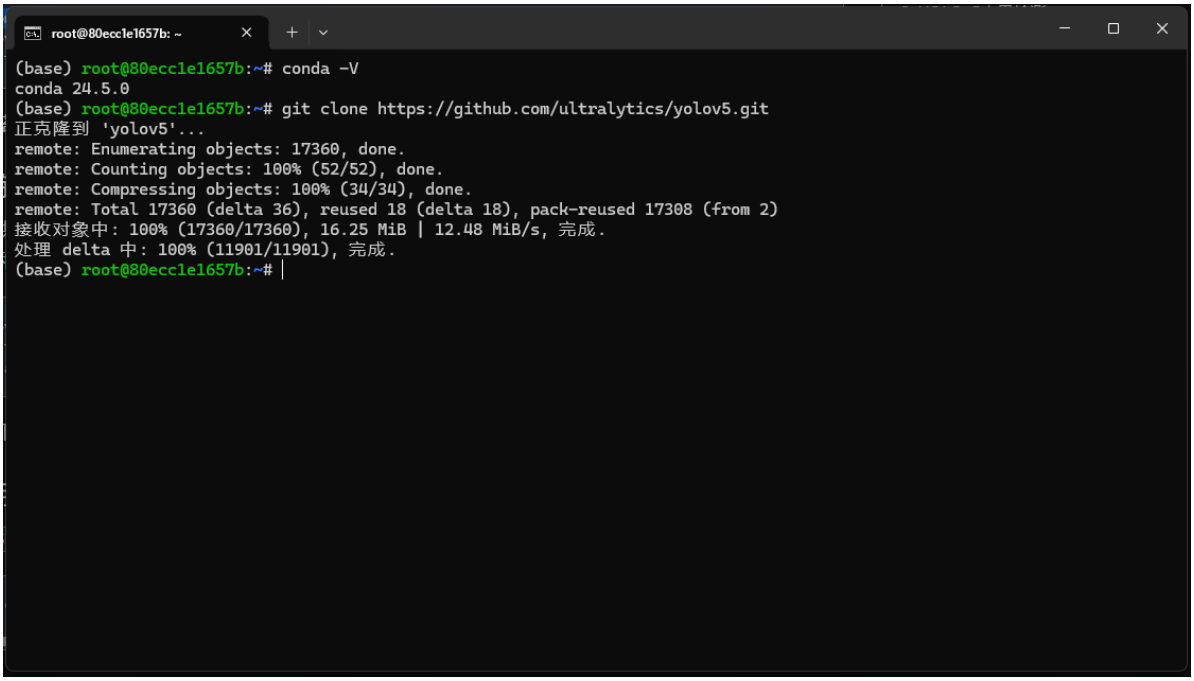
系统已预装了Miniconda3_3.10



下载并安装Yolo v5环境依赖

输入命令

```
git clone https://github.com/ultralytics/yolov5.git
```

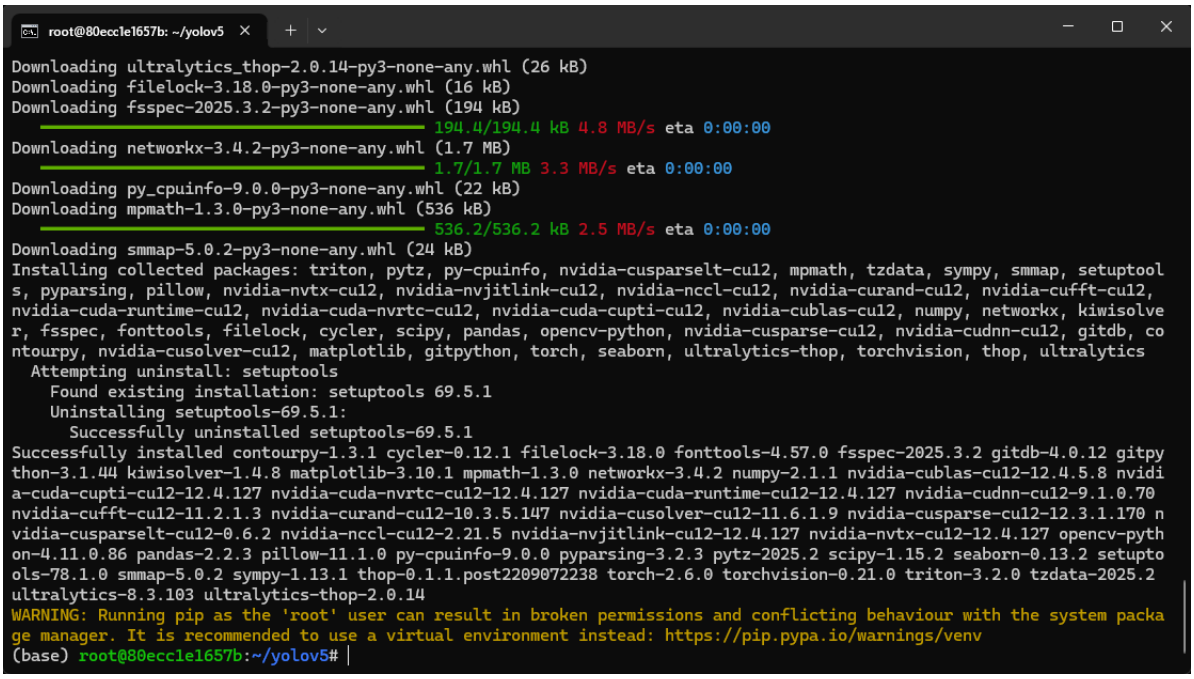


执行

```
cd yolov5
```

```
pip install -r requirements.txt
```

> 这部分文件内容较多，请耐心等待

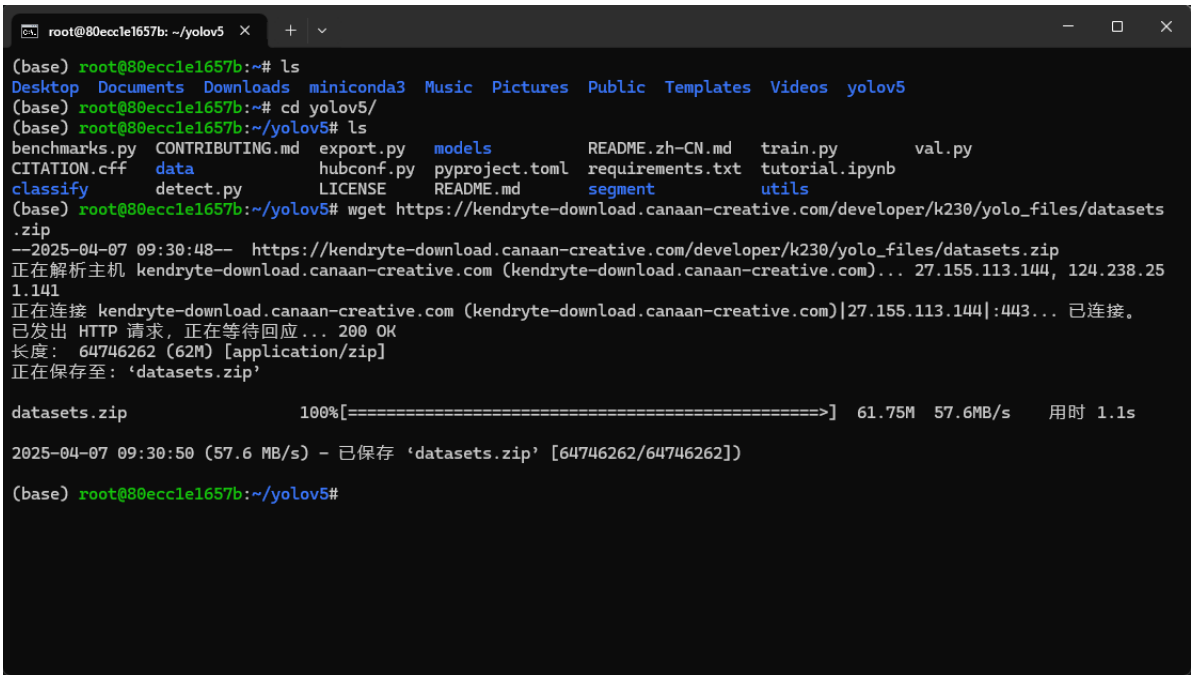


准备训练数据

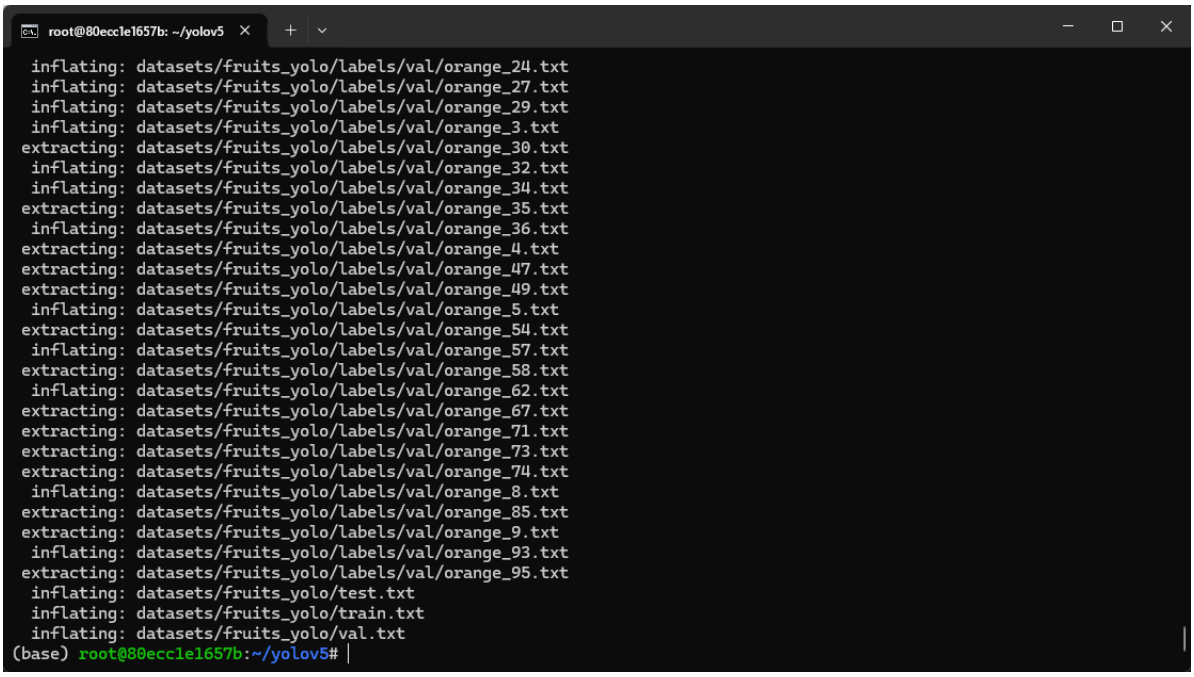
> 这里我们以嘉楠科技官方提供的训练集作为示例数据
>
> 请下载提供的示例数据集，示例数据集中包含以三类水果（apple，banana，orange）为场景分别提供了分类、检测和分割数据集。将数据集解压到 `yolov5` 目录下，请使用 `fruits_cls` 作为水果分类任务的数据集。

我们执行下述的代码

```
wget https://kendryte-download.canaan-creative.com/developer/k230/yolo_files/datasets.zip
```




```
unzip datasets.zip
```



使用YOLOv5训练水果分类模型

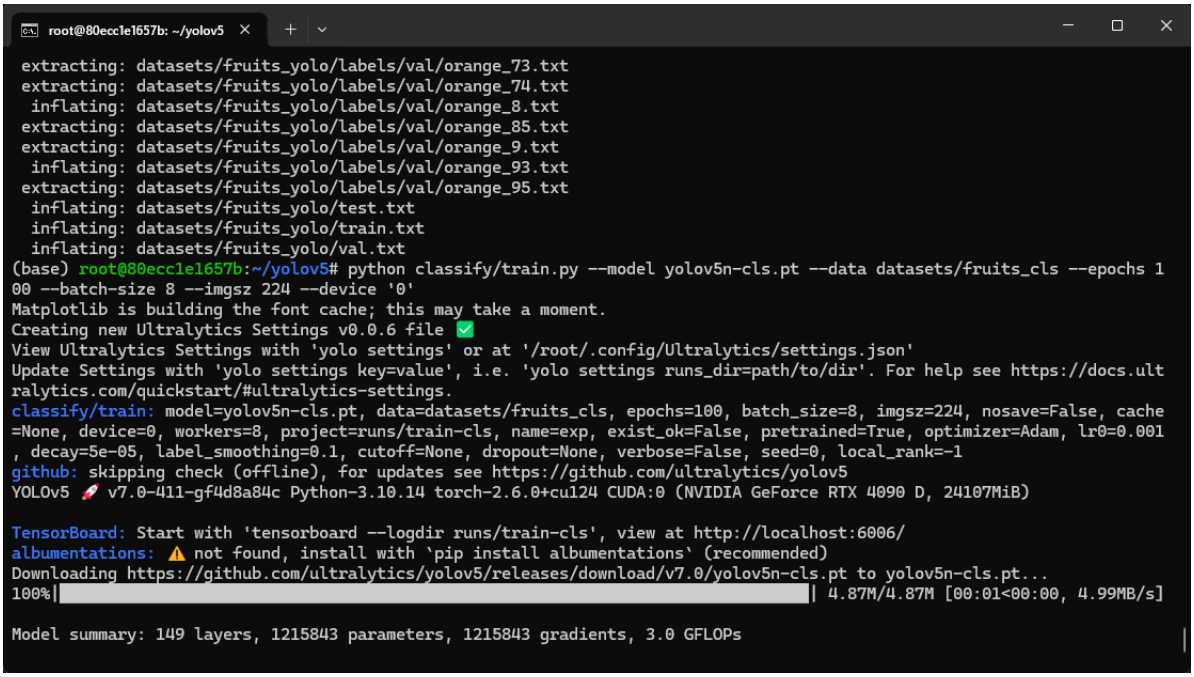
> 在 `yolov5` 目录下执行命令，使用 `yolov5` 训练三类水果分类模型：

我们进入yolov5目录（示例中为~/yolov5）

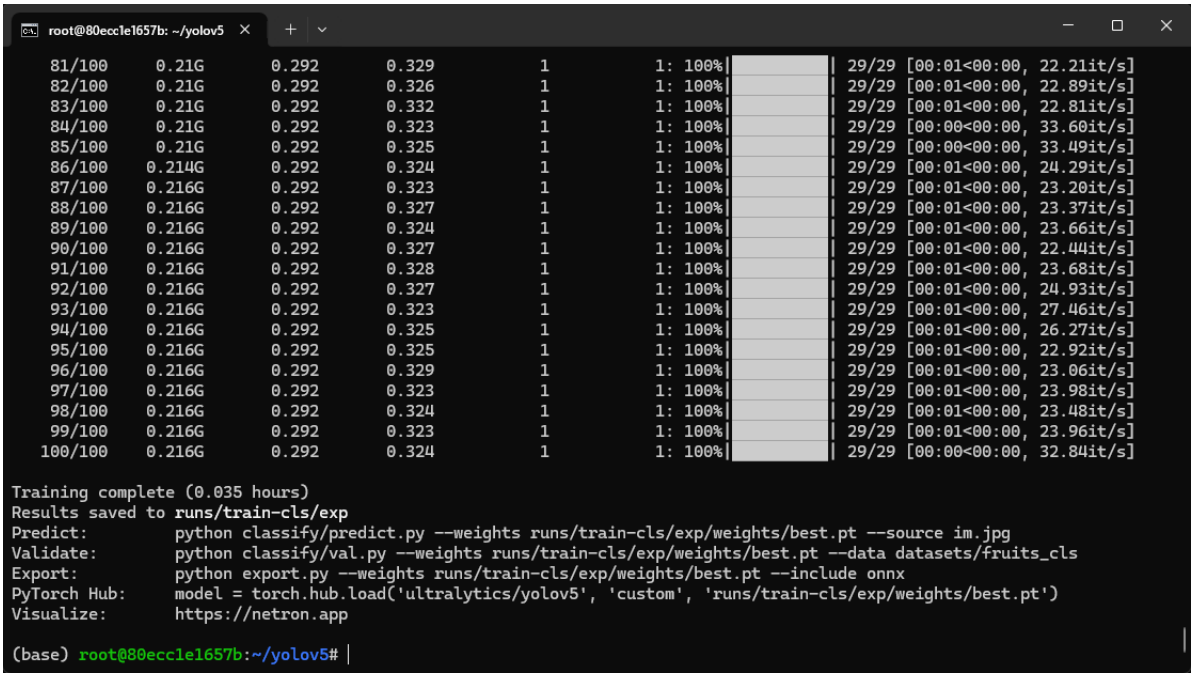
执行下述代码开始训练

```
python classify/train.py --model yolov5n-cls.pt --data datasets/fruits_cls --epochs 100 --batch-size 8 --imgsz 224 --device '0'
```

出现类似于下图的界面，则说明开始训练了



训练成功结束后，输出如下



将训练的模型转为K230可以识别的kmodel

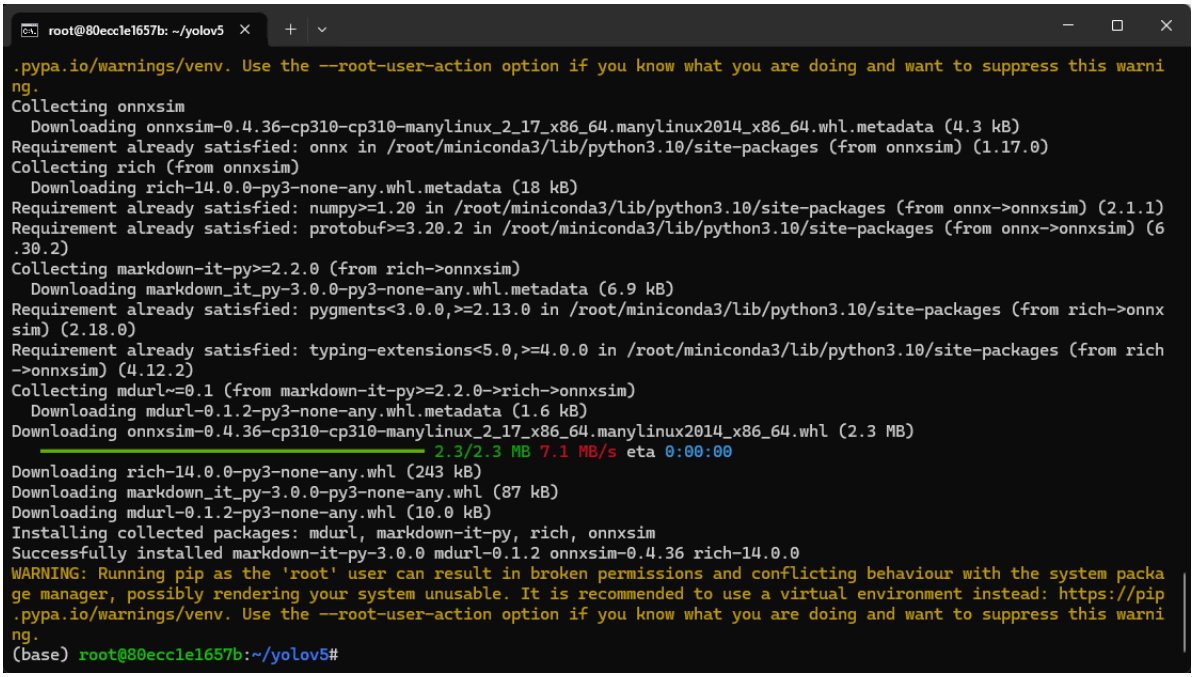
我们用yolov5训练出来的模型是pt后缀的best.pt

为了在K230上使用，我们需要将其转化为kmodel格式的模型

下载转换工具

执行下述代码，安装需要用到的工具

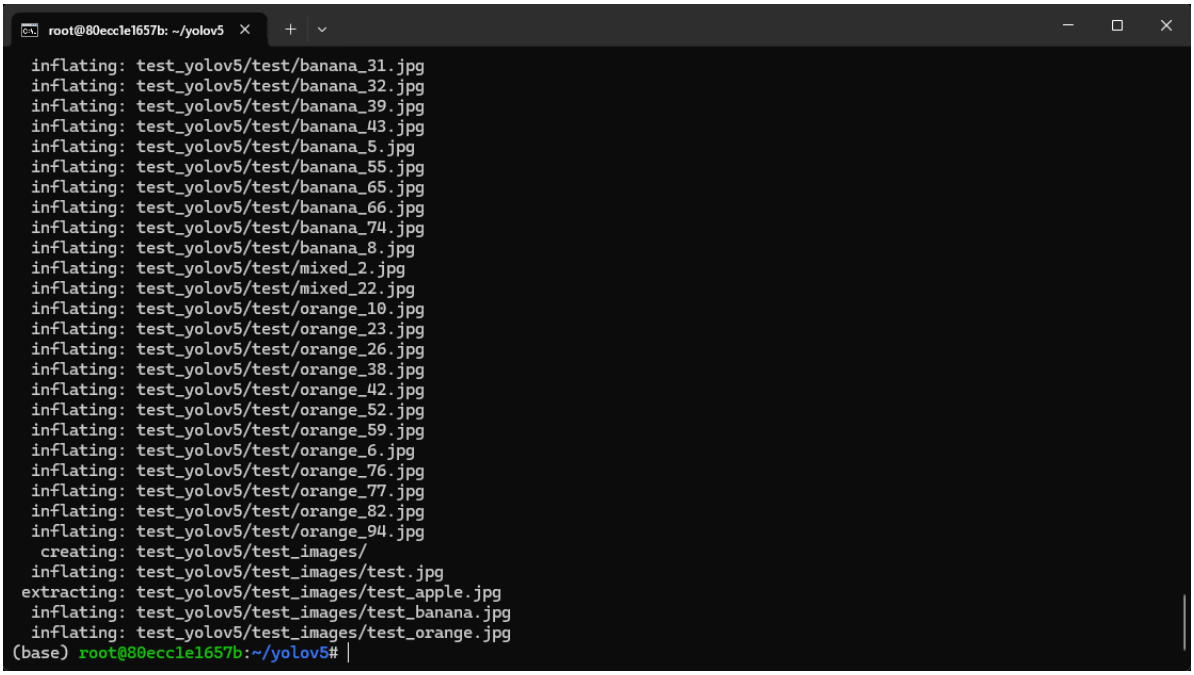
```
sudo apt-get install -y dotnet-sdk-7.0
pip install --upgrade pip
pip install nncase==2.9.0
pip install nncase-kpu==2.9.0
pip install onnx
pip install onnxruntime
pip install onnxsim
```



下载脚本工具，将模型转换脚本工具 `test_yolov5.zip` 解压到 `yolov5` 目录下；

```
wget https://kendryte-download.canaan-
creative.com/developer/k230/yolo_files/test_yolov5.zip
```

```
unzip test_yolov5.zip
```



开始进行模型格式转换

第一步：pt格式转为onnx格式

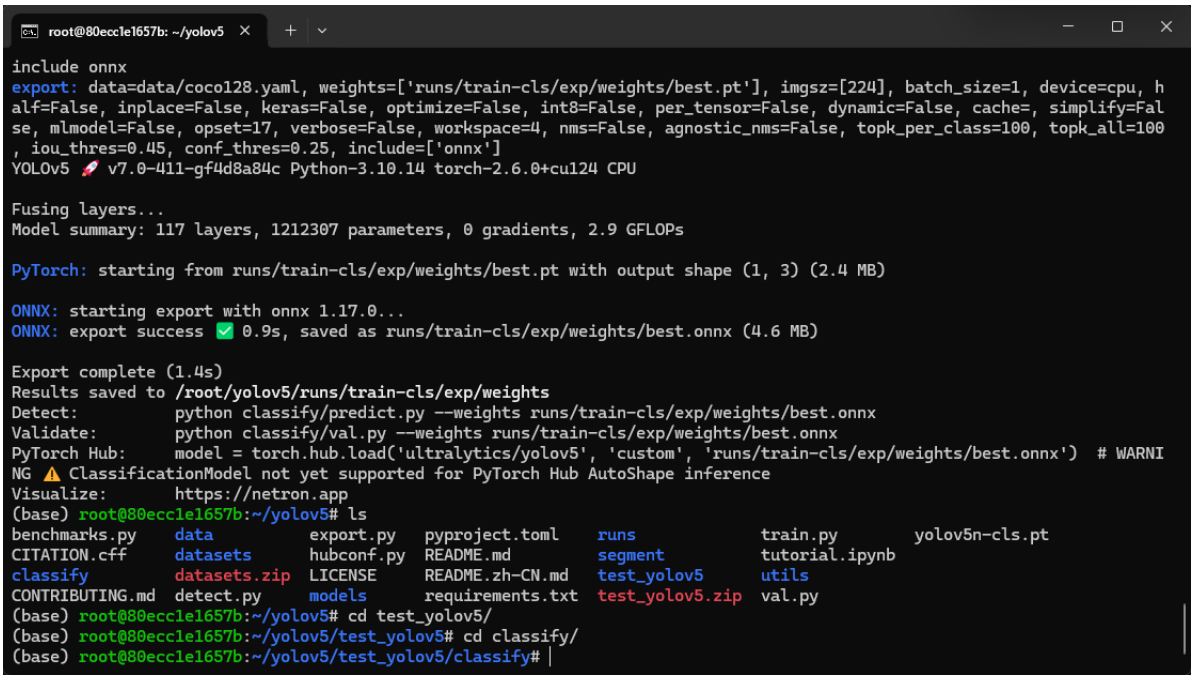
```
python export.py --weight runs/train-cls/exp/weights/best.pt --imgsz 224 --batch
1 --include onnx
```

注意，此处的 "runs/train-cls/exp/weights/best.pt" 是我们训练出来的pt模型的路径。

> 如果前面的步骤完全是按照教程来做的，那么此处路径不需要修改

第二步：进入test_yolov5/classify目录

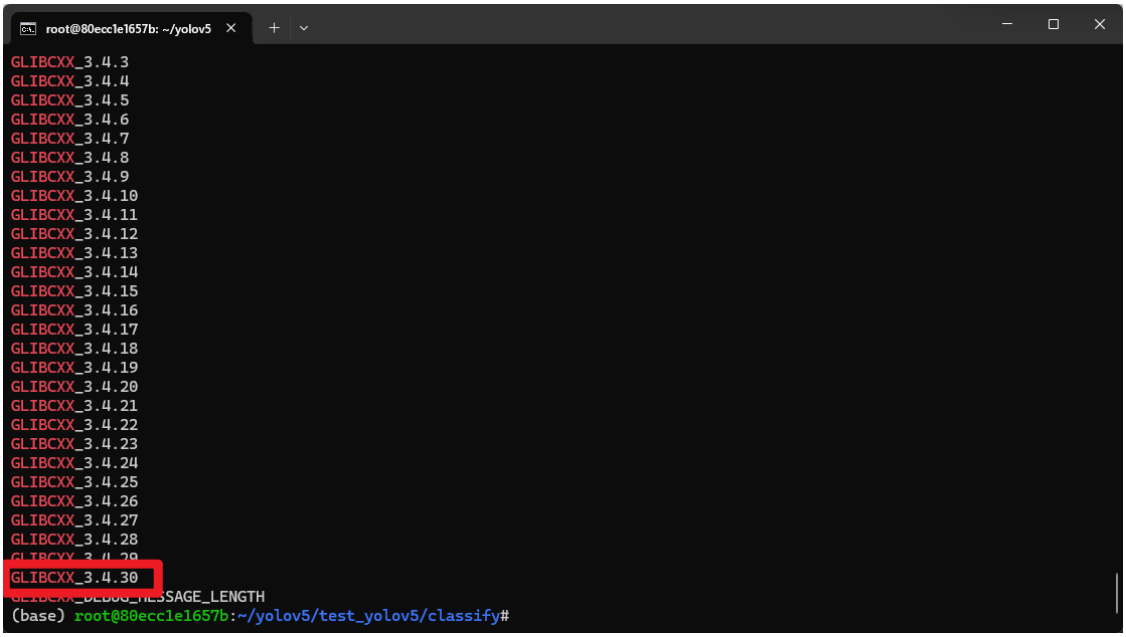
```
cd test_yolov5/classify
```



执行模型转换指令

```
python to_kmodel.py --target k230 --model ../../runs/train-cls/exp/weights/best.onnx --dataset ../test --input_width 224 --input_height 224 --ptq_option 0
```

> 注：此时可能出现报错：RuntimeError: /root/miniconda3/bin/../lib/libstdc++.so.6: version `GLIBCXX_3.4.30' not found (required by /usr/lib/dotnet/host/fxr/7.0.19/libhostfxr.so): Success
>
> 解决方案：
>
> 检查GLIBCXX_3.4.30是否存在
>
> ```
> strings /usr/lib/x86_64-linux-gnu/libstdc++.so.6 | grep GLIBCXX
> ```
>
> 如果输出中有，则说明不是真的缺失了该文件，而是虚拟环境中的libstdc++版本比系统中libstdc++这个库的版本要低，



我们此时只需执行如下指令即可

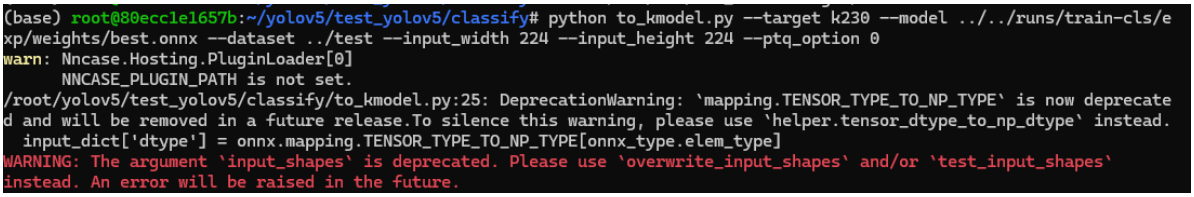
将虚拟环境中的libstdc++.so.6文件删除（或重命名为其他名称，这里我们重命名为libstdc++.so.6.old）

```
mv /root/miniconda3/bin/../lib/libstdc++.so.6
/root/miniconda3/bin/../lib/libstdc++.so.6.old
```

然后建立软连接，让虚拟环境调用的libstdc++.so.6指向系统自带的libstdc++.so.6即可

```
ln -s /usr/lib/x86_64-linux-gnu/libstdc++.so.6 libstdc++.so.6
```

运行结束转换指令结束后，输出如下



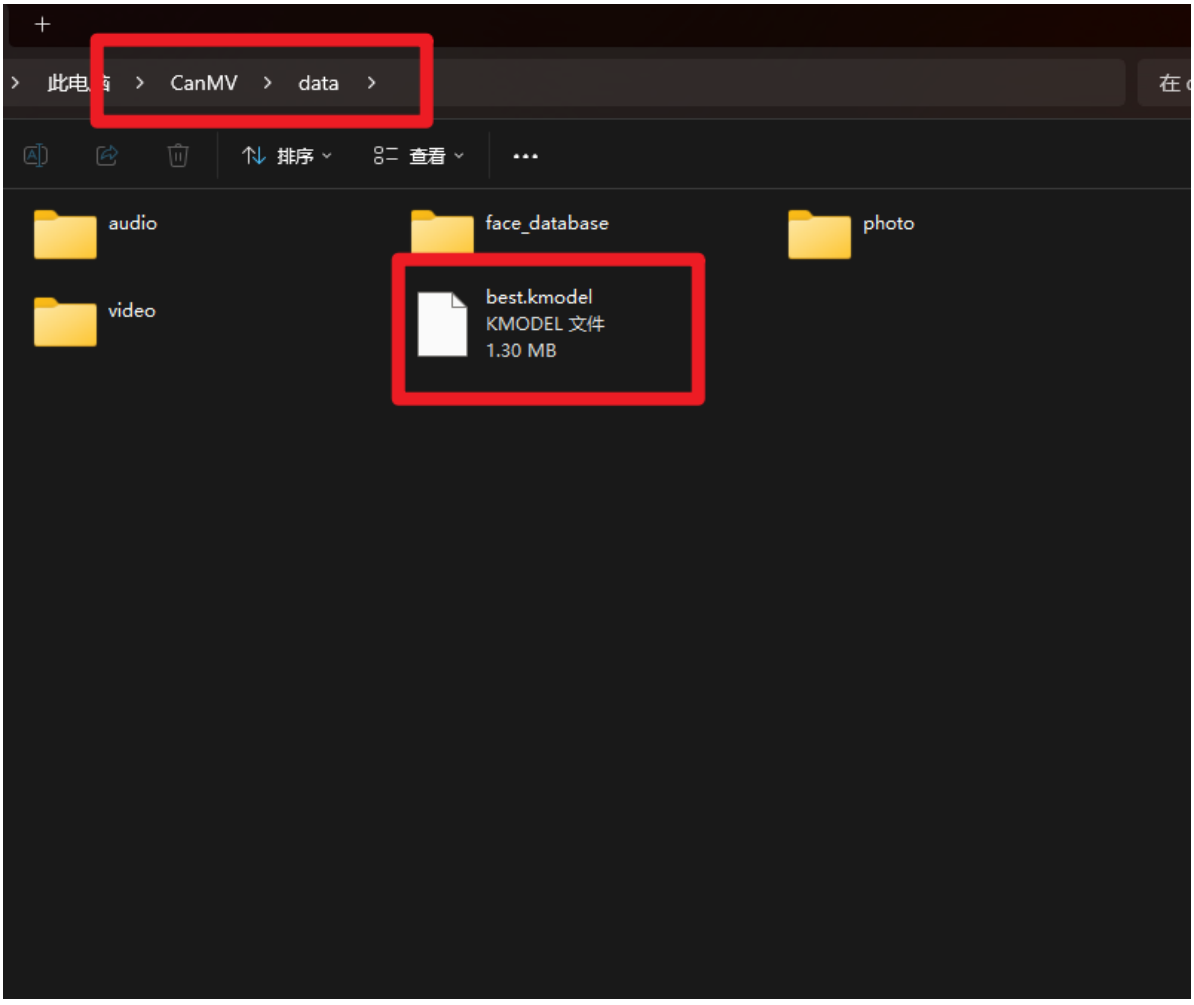
WARNING这里可以忽略

生成的模型在~/yolov5/runs/train-cls/exp/weights这个目录中

```
root@80ecc1e1657b: ~/yolov5
in/../lib/libstdc++.so.6.old
(base) root@80ecc1e1657b:~/yolov5/test_yolov5/classify# ln -s /usr/lib/x86_64-linux-gnu/libstdc++.so.6 libstdc++.so.6
(base) root@80ecc1e1657b:~/yolov5/test_yolov5/classify# python to_kmodel.py --target k230 --model ../../runs/train-cls/exp/weights/best.onnx --dataset ../test --input_width 224 --input_height 224 --ptq_option 0
warn: Ncase.Hosting.PluginLoader[0]
      NNCASE_PLUGIN_PATH is not set.
/root/yolov5/test_yolov5/classify/to_kmodel.py:25: DeprecationWarning: `mapping.TENSOR_TYPE_TO_NP_TYPE` is now deprecated and will be removed in a future release.To silence this warning, please use `helper.tensor_dtype_to_np_dtype` instead.
  input_dict['dtype'] = onnx.mapping.TENSOR_TYPE_TO_NP_TYPE[onnx_type.elem_type]
WARNING: The argument 'input_shapes' is deprecated. Please use 'overwrite_input_shapes' and/or 'test_input_shapes' instead. An error will be raised in the future.
(base) root@80ecc1e1657b:~/yolov5/test_yolov5/classify# cd ../../
(base) root@80ecc1e1657b:~/yolov5# ls
benchmarks.py    data          export.py   pyproject.toml    runs            train.py        yolov5n-cls.pt
CITATION.cff     datasets      hubconf.py  README.md         segment         tutorial.ipynb
classify         datasets.zip  LICENSE     README.zh-CN.md   test_yolov5     utils
CONTRIBUTING.md  detect.py     models      requirements.txt  test_yolov5.zip  val.py
(base) root@80ecc1e1657b:~/yolov5# cd runs/
(base) root@80ecc1e1657b:~/yolov5/runs# ls
train-cls
(base) root@80ecc1e1657b:~/yolov5/runs# cd train-cls/
(base) root@80ecc1e1657b:~/yolov5/runs/train-cls# ls
exp
(base) root@80ecc1e1657b:~/yolov5/runs/train-cls# cd exp/
(base) root@80ecc1e1657b:~/yolov5/runs/train-cls/exp# ls
opt.yaml  results.csv  test_images.jpg  train_images.jpg  weights
(base) root@80ecc1e1657b:~/yolov5/runs/train-cls/exp# cd weights/
(base) root@80ecc1e1657b:~/yolov5/runs/train-cls/exp/weights# ls
best.kmodel  best.onnx  best.pt  last.pt
(base) root@80ecc1e1657b:~/yolov5/runs/train-cls/exp/weights#
```

名称叫做 best.kmodel

我们将这个 best.kmodel 复制到我们的K230的data目录下



然后打开CanMV IDE，复制下面的代码，点击运行

```
from libs.PipeLine import PipeLine, ScopedTiming
from libs.YOLO import YOLOv5
import os,sys,gc
import ulab.numpy as np
```

```
import image

if __name__=="__main__":

    rgb888p_size=[1280,720]
    display_size=[640,480]
    # 模型路径
    kmodel_path="/data/best.kmodel"
    labels = ["apple","banana","orange"]
    confidence_threshold = 0.5
    model_input_size=[224,224]
    # 初始化PipeLine

 pl=PipeLine(rgb888p_size=rgb888p_size,display_size=display_size,display_mode="l
cd")
    pl.create()
    # 初始化YOLOv5实例

 yolo=YOLOv5(task_type="classify",mode="video",kmodel_path=kmodel_path,labels=la
bels,rgb888p_size=rgb888p_size,model_input_size=model_input_size,display_size=di
splay_size,conf_thresh=confidence_threshold,debug_mode=0)
    yolo.config_preprocess()
    try:
        while True:
            os.exitpoint()
            with ScopedTiming("total",1):
                # 逐帧推理
                img=pl.get_frame()
                res=yolo.run(img)
                yolo.draw_result(res,pl.osd_img)
                pl.show_image()
                gc.collect()
    except Exception as e:
        sys.print_exception(e)
    finally:
        yolo.deinit()
        pl.destroy()
```

我们测试一下效果

识别苹果


apple 0.934
直方图
平均数 113
最小 0
平均数 92
最小 12
平均数 89
最小 0

识别香蕉


banana 0.936
直方图

识别橘子



可以看到这个例程能成功运行了

其它注意事项

1. 环境配置、Yolov5环境依赖以及模型转换工具仅需下载一次，后续训练时不再需要重新下载
2. 如果执行pip下载的时候速度远低于实际网络带宽，请检查一下是否有配置pip镜像源
3. 当前数据集训练出来的模型可能会把橘子/苹果以外的非香蕉的物体识别为香蕉，这是正常情况，如果需要更加准确的模型，可以准备一个数据量更大的训练集进行训练